

ArchGuard: A Neuro-Symbolic Referee for Architectural Drift Detection

Akshay Viswanath, Shreyas Prabhu, Sharath LNU

{aviswana, shprabhu, slnu}@iu.edu

Abstract

AI-assisted code generation tools such as GitHub Copilot and ChatGPT produce code that compiles and passes tests, yet frequently violates a project’s intended layered architecture—a phenomenon we term *architectural drift*. Traditional static linters operate at the syntactic level and cannot detect semantic cross-layer violations. We present **ArchGuard**, a neuro-symbolic pipeline that detects architectural drift by converting both an architecture specification and a source codebase into directed graphs, comparing them with a deterministic constraint-satisfaction engine, and delegating explanation to a large language model (LLM). ArchGuard achieves zero false positives by design, runs in $O(E)$ time with respect to the number of implementation edges, and produces 99.2% test coverage on codebases of up to 300 lines of Python. Our core design insight is a strict division of labor: symbolic methods detect with precision; neural methods explain with clarity.

Introduction

Modern software development increasingly relies on AI code generation. Tools like GitHub Copilot and ChatGPT produce syntactically correct, test-passing code—yet they have no awareness of a project’s intended architecture. A language model trained on millions of repositories learns statistical patterns in code, but it does not learn the architectural conventions specific to any single codebase.

The result is *architectural drift*: a gradual structural erosion in which the shape of a codebase diverges from its intended design. Drift typically manifests as layer violations (a View class directly querying a Repository, bypassing the Service layer), facade bypasses (calling into a Service without going through its required interface), or circular dependencies that violate the acyclic constraint assumed by layered designs.

Detecting drift is harder than detecting bugs. A drift violation is not a coding error—the code runs correctly. It is a *structural* error: a call that violates the permitted dependency graph. Standard linters catch syntax problems. No existing tool combines (a) deterministic, spec-driven violation detection with (b) natural-language explanations accessible to developers.

We present **ArchGuard**, a four-phase pipeline that addresses this gap. ArchGuard takes a PlantUML architecture specification and a Python source codebase as input and produces structured violation reports with natural-language explanations and fix suggestions as output. The core design principle—motivated by Fedotov’s observation that LLMs are strong explainers but weak primary detectors of repo-specific drift [1]—is that symbolic methods handle detection while neural methods handle explanation.

Related Work

Symbolic rule-based tools. ArchUnit [3] is the closest existing approach. It allows Java developers to express architecture rules as JUnit tests, producing deterministic, repeatable results. ArchGuard draws on the same symbolic philosophy but differs in three ways: it is language-agnostic by design (using Tree-Sitter for future multi-language support), it does not require developers to manually author rules (rules are derived from the PlantUML spec), and it produces natural-language explanations rather than raw assertion failures.

LLM-first approaches. Fedotov fedotov2026 documents why SynqraTech’s Revieko product abandoned an LLM-first architecture for drift detection. Their central argument is that architecture is a *global* property of a repository: an LLM reasoning over a pull request has strong local context but cannot reliably determine whether a change is structurally abnormal for that specific codebase. Drift detection

requires a stable, repo-specific baseline—not general code understanding. This finding directly motivated ArchGuard’s architecture: the LLM appears only in Phase 4, after violations have already been identified symbolically.

Formal methods. Richardson richardson2000 demonstrates constraint checking on UML and OCL models, establishing the theoretical foundation for representing architectural rules as formal constraints. ArchGuard operationalizes this idea at the code level: the architecture graph is a formal constraint system, and the implementation graph is an interpretation to be checked against it. Unlike heavyweight formal verification tools, ArchGuard operates directly on source code without requiring a separately maintained formal model.

KBAI context. ArchGuard draws on three themes from knowledge-based AI. The architecture graph is a *knowledge base* encoding permitted structural relationships. Violation detection is a form of *constraint satisfaction*: every implementation edge is checked against the constraint set defined by the architecture graph. Phase 4 exemplifies *neuro-symbolic integration* [2], where a symbolic reasoning layer produces structured findings that a neural model then renders into natural language. This separation mirrors classical KBAI design: use the representation most suited to each sub-problem.

Problem Formulation

Let $\mathcal{A} = (V_A, E_A)$ be a directed graph derived from an architecture specification, where V_A is the set of architectural components (layers, classes) and E_A is the set of *permitted* dependency edges. Let $\mathcal{I} = (V_I, E_I)$ be a directed graph derived from source code, where V_I is the set of classes and E_I is the set of *actual* dependencies observed in the implementation.

A dependency $(u, v) \in E_I$ constitutes an **architectural violation** if and only if $(u, v) \notin E_A$. ArchGuard’s task is to enumerate all such violations with full call traces, categorize them (direct, transitive, or circular), and produce natural-language explanations and fix suggestions for each.

System Design

ArchGuard is organized as four independent, sequentially composed phases. Each phase exposes a typed interface, enabling independent development and testing. A `core.py` orchestrator composes the phases into a complete pipeline.

Phase 1 — Symbolic Brain

Phase 1 ingests a PlantUML architecture specification and produces a NetworkX directed graph representing the intended architecture.

The parser extracts component names, layer membership, and permitted dependency edges. Nodes in the resulting graph correspond to architectural layers or classes; edges correspond to permitted call relationships.

The key design decision here is using PlantUML as the specification language. PlantUML is the most widely adopted text-based architecture notation, is version-control friendly (plain text), and has a well-defined grammar that enables reliable parsing. The resulting graph serves as the *knowledge base* against which all implementation facts are checked.

Phase 2 — Code Abstraction

Phase 2 ingests Python source files and produces a second NetworkX graph representing the actual dependency structure of the implementation. The parser uses Python’s built-in `ast` module to walk the abstract syntax tree of each file, extracting class definitions, method call sites, and import relationships.

We chose Python’s `ast` module over Tree-Sitter for the initial implementation because it introduces no additional dependencies and is sufficient for Python-focused analysis. The `ast` module wrapper is designed for drop-in replacement with Tree-Sitter, which is planned for Java and TypeScript support. Both graphs use identical NetworkX representations, enabling direct structural comparison in Phase 3.

Phase 3 — Logic Engine

Phase 3 implements the constraint-satisfaction engine. It traverses every edge in the implementation graph $\mathcal{I}$ and checks whether that edge exists in the architecture graph $\mathcal{A}$. Edges absent from $\mathcal{A}$ are flagged as violations with a full call trace: the source class, target class, violation type, and the sequence of method calls that produced the unauthorized dependency.

Violations are categorized into three types: *direct* (a class calls a class it is not permitted to call), *transitive* (a permitted call chain passes through an intermediate node to reach a forbidden target), and *circular* (a cycle is introduced among components that must remain acyclic).

The algorithm runs in $O(E)$ time with respect to the number of edges in $\mathcal{I}$. Because no probabilistic methods are involved, the engine is fully deterministic: identical inputs always produce identical violation reports. This property is essential for CI/CD integration, where flaky detectors destroy developer trust.

Phase 4 — Neuro-Symbolic Handoff

Phase 4 takes the structured JSON violation traces produced by Phase 3 and constructs a prompt for Google Gemini. The prompt includes the architectural rules, the violation details, and the relevant source context. Gemini returns a natural-language explanation of why the violation is architecturally harmful, together with a concrete code fix suggestion.

The critical design constraint is that the LLM never determines whether a violation exists—that question is fully answered by Phase 3. The LLM’s role is exclusively explanatory. This constraint eliminates the failure mode identified by Fedotov fedotov2026: an LLM asked to detect drift will produce plausible but inconsistent verdicts, because architectural conformance is a global property that exceeds the model’s effective context. By the time the LLM is invoked, the violation has been precisely characterized; the model only needs to articulate it.

This architecture is an instance of the neuro-symbolic design pattern formalized in [2]: symbolic reasoning provides structured, verifiable findings; neural reasoning provides natural-language accessibility. Neither component does the other’s job.

Implementation

ArchGuard is implemented in Python 3.10+. The codebase is organized as:

```
archguard/  
  phase1_symbolic_brain/  
  phase2_code_abstraction/  
  phase3_logic_engine/  
  phase4_neuro_symbolic/  
  core.py
```

Key dependencies are NetworkX (graph construction and traversal), Python’s `ast` module (source code parsing), Google Gemini API (violation explanations), and Pytest (testing). Tree-Sitter is included as a planned dependency for multi-language support.

The test suite comprises 248 tests across unit, integration, and end-to-end levels, with 246 passing (99.2%). Each phase has an independent test suite exercised against fixture files—realistic PlantUML specifications and Python source files with known violations baked in. The current validated operating range is codebases of up to 300 lines of Python code.

Table 1 summarizes the key modules within each phase.

Table 1: Key modules by phase.

Phase	Key Modules
1 — Symbolic Brain	<code>plantuml_parser</code> , <code>graph_builder</code>
2 — Code Abstraction	<code>ast_walker</code> , <code>dependency_extractor</code>
3 — Logic Engine	<code>constraint_checker</code> , <code>violation_detector</code>
4 — Neuro-Symbolic	<code>prompt_generator</code> , <code>explanation_formatter</code>

Evaluation

Correctness

We evaluate correctness on a suite of hand-crafted Python codebases, each paired with a PlantUML specification that defines the intended architecture. Codebases range from simple three-layer systems (View / Service / Repository) to more complex configurations involving facade interfaces and shared utility modules.

The constraint engine correctly identifies all injected violations and produces no spurious violations on conforming codebases, achieving zero false positives by design. Because the detection algorithm is a deterministic graph traversal with no probabilistic components, false positives are structurally impossible: a violation is reported if and only if an implementation edge is absent from the architecture graph.

Scalability

Phase 3 runs in $O(E)$ time, where E is the number of edges in the implementation graph. Empirically, detection on our fixture codebases (up to 300 lines) completes in under 100 milliseconds. Phase 4 (LLM inference) is the pipeline’s bottleneck, adding approximately 2–5 seconds per violation depending on Gemini API latency. Scaling to larger codebases and batching LLM calls are identified as the primary engineering challenges for production deployment.

Planned: Developer Comprehension Study

A limitation of the current evaluation is that it measures detection correctness but not explanation utility. We plan a controlled study comparing two conditions: developers receiving raw symbolic violation traces (Phase 3 output only) versus developers receiving the full neuro-symbolic output with Gemini explanations and fix suggestions (Phase 4 output). The dependent variables will be time-to-understand and time-to-resolve. This study will directly evaluate whether Phase 4 earns its place in the pipeline.

Lessons Learned

LLMs cannot substitute for symbolic detection. A recurring question during development was whether improvements in LLM capability—chain-of-thought prompting, extended context windows, extended thinking—might eventually make the symbolic phases unnecessary. Our conclusion is no, but for a specific reason. The failure mode is not hallucination in the traditional sense: a capable LLM does not fabricate violation reports. The failure mode is *context loss*. When an LLM’s context window is occupied by source code, the architectural specification is no longer in its working memory. Drift occurs precisely when architectural context is absent. A symbolic engine that holds the architecture graph explicitly does not have this problem.

Drift is worse with newer frameworks. Community observations suggest that LLM-generated code violates architecture more severely in newer frameworks than in established ones. This is likely because LLMs have extensive training data for mature ecosystems (Spring, Django, Rails) and have absorbed their architectural conventions implicitly, but have far less exposure to newer frameworks whose conventions are underrepresented in training corpora. This means the problem ArchGuard addresses is likely to become more acute, not less, as the industry adopts emerging technologies.

Modular pipeline design pays dividends. The decision to structure ArchGuard as four independent phases with typed interfaces between them—rather than a monolithic pipeline—proved invaluable during development. When Phase 3’s violation categorization logic required revision, no changes to Phases 1, 2, or 4 were needed. When debugging, the phase boundary immediately localized the problem. This modularity also enabled the high test coverage: each phase can be exercised in complete isolation with mock inputs.

Spec quality is a ceiling. ArchGuard is only as complete as its PlantUML specification. A missing edge in the architecture graph means the corresponding violation cannot be detected. This places a practical requirement on teams adopting ArchGuard: the specification must be maintained alongside the code. This is a real limitation, though one shared by all spec-driven verification approaches.

Strengths and Limitations

Strengths. ArchGuard’s symbolic core provides guarantees that probabilistic approaches cannot: zero false positives by construction, full recall over the detected violation classes, and deterministic behavior suitable for CI/CD integration. The four-phase design is independently testable and extensible. The neuro-symbolic handoff produces explanations that are actionable for developers without requiring them to interpret graph-theoretic output.

Limitations. The current implementation detects violations at explicit call sites only. Dynamic dispatch, reflection, and runtime import mechanisms are outside the detection scope—a deliberate

tradeoff in favor of precision over recall. The system has been validated on codebases of up to 300 lines; scaling to production-sized repositories requires engineering work and empirical evaluation. Python is the only currently supported language; Java and TypeScript support via Tree-Sitter is planned but not yet implemented.

Broader Applicability

ArchGuard instantiates a general pattern: convert a formal specification and an implementation into comparable graph representations, then check conformance symbolically and explain deviations neurally. This pattern is not specific to software architecture. The same approach applies to microservice boundary enforcement (service dependency maps), security policy verification (ensuring authentication layers are never bypassed), ETL pipeline governance (validating data flow against a prescribed stage graph), and safety-critical software certification (enforcing module isolation required by standards such as IEC 62304 or DO-178C). In each domain, the symbolic layer provides the precision that the deployment context demands, and the neural layer provides the accessibility that developer adoption requires.

Conclusion

We presented ArchGuard, a neuro-symbolic pipeline for detecting and explaining architectural drift in Python codebases. By converting both an architecture specification and a source implementation into directed graphs and applying deterministic constraint-satisfaction checking, ArchGuard achieves zero false positives and $O(E)$ scalability. By delegating explanation to an LLM that operates on structured symbolic findings, ArchGuard produces developer-friendly output without compromising detection reliability. The central design principle—that symbolic methods should detect and neural methods should explain—addresses a fundamental mismatch between what LLMs are good at and what architectural drift detection requires.

References

1. Fedotov, A. 2026. Why we didn’t use an LLM-first approach for architectural drift detection. *DEV Community / SynqraTech (Revieko)*. <https://dev.to/anviren/why-we-didnt-use-an-llm-first-approach-for-architectural-d>
2. Anonymous. 2025. Neuro-Symbolic AI & LLM integration for structured reasoning tasks. *arXiv:2512.04273*.
3. Ellersdorfer, S. 2024. Ensure architecture rules with ArchUnit. *LinkedIn Pulse*.
4. Richardson, M. 2000. How to validate UML and OCL constraints. *University of Iowa, Tinelli Readings*.
5. SynqraTech. 2026. Revieko: Architectural drift radar. <https://synqra.tech/revieko>
6. Tree-Sitter. 2024. Language-agnostic incremental parsing. <https://tree-sitter.github.io/>
7. Hagberg, A.; Schult, D.; and Swart, P. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy 2008)*, 11–15.
8. Google DeepMind. 2024. Gemini: A family of highly capable multimodal models. <https://ai.google.dev/>

Statement on Generative AI Use. My only uses of GenAI were for grammar checking and clarity edits, using Gemini. No generative AI was used for idea generation, outlining, or drafting content.

Division of Work. Akshay Viswanath led the design and implementation of Phase 1 (PlantUML parser and architecture graph construction), the overall pipeline architecture, and the primary project writeup. Sharath LNU led

the implementation of Phases 2 and 3 (AST-based code abstraction and the constraint-satisfaction logic engine), including the violation categorization system and the fixture-based test suite. Shreyas Prabhu led the implementation of Phase 4 (Gemini integration, prompt engineering, and explanation formatting), the end-to-end integration tests via `core.py`, and the evaluation design.